Final Exam Preparation: Level 5 Descriptions

Based on the final exam pool detailed in PLAN_2.pdf, here is the breakdown for the Level 5 questions. These are complex, multi-step parsing and logic problems.

Note: All of the Level 5 tasks are **Programs** (they require a `main`, `argc`, `argv`, and print to standard output).

Level 5 (Advanced Parsing & Logic)

1. `brainfuck` (Program)

1. **Description:** Write a Brainfuck language interpreter program. The program takes a single string representing the source code and evaluates it. The Brainfuck environment consists of an array of 2048 bytes (initialized to 0) and a pointer to the first byte. The language has the following commands:
    - `>` : Increment the pointer.
    - `<` : Decrement the pointer.
    - `+` : Increment the byte pointed to.
    - `-` : Decrement the byte pointed to.
    - `.` : Print the byte pointed to on standard output.
    - `[` : If the byte pointed to is 0, jump past the matching `]`.
    - `]` : If the byte pointed to is non-zero, jump back to the matching `[`. Any other character in the string is ignored. If no arguments or more than one argument are given, just print a newline.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**
    - Input: `./brainfuck "++++++++++[>+++++++>++++++++++>+++>+<<<<-]>++.>+.+++++++..+++.>++.<<+++++++++++++++.>.+++.------.--------.>+.>."`

- Output: `Hello World!\n`
- Input: `./brainfuck "+++++[>++++[>++++H>+++++i<<<-]>>>++\n<<<<<-]a>>>>\n.<<<p>>>>\n.p<<<.y>>>>>\n.<l<<<<<<.o>>>>>\n.<<<<<w>>>>\n.e<<<<<l>>>>\n.<<<<<o>>>>\n.w<<<<<r>>>>\n.d>>>>\n.<l<<<<<<.d>>>>>\n.!"`
- Output: `Happy holiday!\n`

2. `brackets` (Program)

1. **Description:** Write a program that takes an undefined number of strings as arguments. For each argument, the program checks if the expression has properly matching brackets: `()`, `[]`, and `{}`.
   - If they are properly closed and nested, print `OK` followed by a newline.
   - If they are not properly closed/nested, print `Error` followed by a newline.
   - If no arguments are passed, it prints nothing (just a newline).
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**
   - Input: `./brackets "(johndoe)"` | Output: `OK\n`
   - Input: `./brackets "([)]"` | Output: `Error\n`
   - Input: `./brackets "{[(0 + 0)(1 + 1)](3*(-1)){()}}"` | Output: `OK\n`
   - Input: `./brackets` | Output: `\n`

3. `check_mate` (Program)

1. **Description:** Write a program that takes rows of a chessboard as arguments and checks if your King is in check.
   - The board is represented by strings forming a square.
   - Pieces are: `P` (Pawn), `B` (Bishop), `R` (Rook), `Q` (Queen), `K` (King).
   - Empty squares are represented by `.`.

- If the King is in check by an enemy piece, print `Success` followed by a newline.
- If the King is not in check, print `Fail` followed by a newline.
- If there is no King, if there are multiple Kings, or if the arguments do not form a perfect square, just print a newline.

2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**
   - Input: `./check_mate '..' '.K'` | Output: `Fail\n`
   - Input: `./check_mate 'R...' '..P.' '.K..' '....'` | Output: `Success\n`
   - Input: `./check_mate '...B.' '.B...' '..K..' '.....' '.....'` | Output: `Success\n`

4. `options` (Program)

1. **Description:** Write a program that takes arguments beginning with `-` and displays the active options as a 32-bit integer format.
   - The valid options are the lowercase letters `a` to `z`.
   - `a` represents the 1st (least significant) bit, `b` is the 2nd bit, ..., `z` is the 26th bit.
   - If an invalid option (e.g., `-2`, `-A`) is passed, print `Invalid Option` followed by a newline.
   - If the `-h` flag is passed anywhere, immediately print `options: abcdefghijklmnopqrstuvwxyz` followed by a newline, ignoring everything else.
   - If no arguments are given, print `options: abcdefghijklmnopqrstuvwxyz` followed by a newline.
   - Output format is four groups of 8 bits separated by spaces.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**

- Input: `./options -abc -c -z` | Output: `00000010 00000000 00000000 00000111\n`
- Input: `./options -j -m` | Output: `00000000 00000000 00010010 00000000\n`
- Input: `./options -hz` | Output: `options: abcdefghijklmnopqrstuvwxyz\n`

5. `rpn_calc` (Program)

1. **Description:** Write a program that takes a string containing an equation in Reverse Polish Notation (RPN) and evaluates it.
    - Valid operators are `+`, `-`, `*`, `/`, and `%`.
    - Numbers and operators are separated by at least one space.
    - If the equation is mathematically correct, print the result.
    - If the equation is invalid, has too many/few operators/operands, or attempts to divide/modulo by zero, print `Error` followed by a newline.
2. **Prototype:** None (Program).
3. **Example Inputs/Outputs:**
    - Input: `./rpn_calc "1 2 * 3 * 4 +"` | Output: `10\n` (which is ((1 * 2) * 3) + 4)
    - Input: `./rpn_calc "1 2 * 3 * 4 + 5"` | Output: `Error\n`
    - Input: `./rpn_calc "4 2 /"` | Output: `2\n`